

# Falando Português

## Trabajo Final — Especialista en Inteligencia Artificial

**Autor:** Ángel Guerra Iglesias

**Racks Academy** · Septiembre de 2026

---

Aplicación móvil para practicar portugués europeo hablado con un tutor de inteligencia artificial, en tiempo real y a cualquier hora.

**Código fuente:** <https://github.com/bolitadenieve69-tech/falando-portugues>. Aplicación React Native con un servidor Python que orquesta un flujo de voz en tiempo real. Se detalla en los apartados 2 y 3, y cuenta con 318 pruebas automáticas. Los datos en bruto de las mediciones y los scripts que los procesan están en el propio repositorio.

**Vídeo demostrativo:** se adjunta con esta memoria.

---

## Índice

1. Identificación del problema u oportunidad
  2. Diseño de la solución con IA
  3. Implementación y validación
  4. Vinculación con competencias adquiridas
  5. Conclusiones y proyección futura
-

# Apartado 1 — Identificación del problema u oportunidad

## Definición del objetivo

Falando Português es una aplicación móvil para mantener conversaciones habladas en portugués europeo con un tutor de inteligencia artificial, en tiempo real y a cualquier hora.

El problema que ataca lo conozco de primera mano, porque aprendo portugués y vivo cerca de la frontera. Al estudiar un idioma, hablar es la única destreza que no puedes practicar solo. La gramática la sacas de un libro, el vocabulario de unas tarjetas, la comprensión de vídeos y podcasts. Pero para producir habla necesitas a alguien enfrente que te escuche, te entienda y te conteste. Y esa persona cuesta dinero, tiene horarios y, para un adulto, impone bastante.

De ahí sale un patrón que cualquiera que haya estudiado idiomas reconoce: gente que entiende bastante y habla poquísimo. Se atascan en la comprensión justamente porque la práctica que les falta es la que no tienen a mano cuando disponen de un rato.

## Impacto negativo de la situación actual

**Coste.** Una hora de conversación con profesor nativo se mueve entre 15 y 30 euros en las plataformas habituales. Media hora diaria son cifras que casi nadie sostiene mes a mes, así que la práctica oral acaba reducida a una o dos sesiones por semana. Es muy poco para automatizar el habla.

**Fricción horaria.** Una clase hay que reservarla y encajarla en la agenda de dos personas. El rato en que un adulto puede ponerse a practicar —de noche, después del trabajo, en un hueco suelto— rara vez coincide con la disponibilidad del profesor.

**Barrera psicológica.** Hablar mal delante de alguien da vergüenza. Y esa vergüenza lleva a evitar precisamente lo que más falta hace, o a refugiarse en cuatro frases seguras en lugar de arriesgarse a construir una nueva.

**El caso concreto del portugués europeo.** Las aplicaciones generalistas sirven portugués de Brasil por defecto: es la variante mayoritaria y es la que traen Duolingo y casi todos sus competidores. Si aprendes para vivir en Portugal, eso significa estudiar palabras que no vas a oír en la calle —*celular* en vez de *telemóvel*, *ônibus* en vez de *autocarro*, *banheiro* en vez de *casa de banho*—, con otra pronunciación y otro tratamiento gramatical. Parte del esfuerzo se va por el desagüe.

## Contexto: sector, usuarios y relevancia

**Sector.** Aprendizaje de idiomas asistido por tecnología. Es un mercado maduro en vocabulario y gramática, pero la conversación hablada sigue resolviéndose casi siempre con profesores humanos. Hasta hace muy poco no había tecnología capaz de sostener un diálogo natural con una latencia que se pudiera tolerar.

**Usuarios afectados.** Adultos que aprenden portugués europeo. Sobre todo:

- Residentes extranjeros en Portugal que necesitan desenvolverse en el día a día.
- Personas con vínculos familiares o profesionales con el país.
- Solicitantes de nacionalidad portuguesa, que tienen que acreditar un nivel A2 mediante el examen CIPLE, una prueba con fecha y con consecuencias reales.

**Relevancia.** Portugal lleva años recibiendo un flujo migratorio sostenido. La reforma de la ley de nacionalidad que entró en vigor en mayo de 2026 amplió el periodo de residencia exigido de cinco a diez años —de tres a siete para ciudadanos de la Unión Europea y de la CPLP— y mantuvo el requisito del A2. En la práctica esto deja un colectivo creciente de personas que van a vivir más años en Portugal antes de obtener la nacionalidad y que, por tanto, necesitan manejarse en portugués europeo durante más tiempo.

## La oportunidad

Hoy convergen tres tecnologías que ya están maduras: reconocimiento de voz en streaming, modelos de lenguaje rápidos y síntesis de voz de calidad. Con ellas se puede construir un interlocutor artificial que responde en menos de dos segundos y medio, está disponible a cualquier hora, no juzga a nadie y cuesta unos céntimos por conversación frente a los 15 o 30 euros de una clase.

No pretendo sustituir al profesor humano. Lo que quiero cubrir es el hueco entre clases: esas horas sueltas en que el estudiante podría estar practicando y no tiene con quién.

---

## Apartado 2 — Diseño de la solución con IA

### Planteamiento

Hay una restricción que condiciona todo lo demás: si el tutor tarda demasiado en contestar, deja de ser una conversación. Por eso fijé desde el principio un requisito no funcional explícito y medible: la latencia percibida, desde que el usuario termina de hablar hasta que empieza a oír la respuesta, no debe pasar de 2,5 segundos.

Repartí ese presupuesto por etapas, y con él en la mano elegí cada componente:

| Etapa                                     | Presupuesto       |
|-------------------------------------------|-------------------|
| Detección de fin de habla y transcripción | ≤ 1.000 ms        |
| Primer token del modelo de lenguaje       | ≤ 500 ms          |
| Inicio del audio sintetizado              | ≤ 800 ms          |
| <b>Total percibido</b>                    | <b>≤ 2.500 ms</b> |

Conviene decir ya que este reparto resultó estar equivocado en una de sus partidas. Lo explico en el apartado 3, cuando aparecen las mediciones.

### Selección de herramientas y justificación

**LiveKit (transporte de audio por WebRTC).** El audio viaja en los dos sentidos de forma continua y con cancelación de eco. Descarté la arquitectura clásica de petición y respuesta sobre HTTP porque obliga a grabar, enviar, esperar y reproducir; con ese ciclo no hay manera de que aquello parezca un diálogo. WebRTC es el estándar de las videollamadas precisamente por eso.

**Deepgram, modelo nova-3 (voz a texto en streaming).** Transcribe mientras el usuario habla, sin esperar a que termine. Lo elegí sobre todo por su detección de fin de intervención, que creía que era la variable que más pesaba en la latencia: decidir demasiado pronto te corta a media frase, y demasiado tarde deja un silencio incómodo.

**Claude Haiku 4.5 (el tutor).** Interpreta lo que dice el usuario, contesta en portugués europeo y corrige los errores gramaticales. Escogí a propósito un modelo rápido y barato en lugar de uno mayor: hablando, medio segundo de espera molesta más de lo que ayuda una respuesta un poco más elaborada. Es una decisión de diseño y no una limitación de presupuesto.

**ElevenLabs, modelo multilingüe (texto a voz).** Pone la voz del tutor. Es de los pocos proveedores con voces auténticas de portugués de Portugal y no de Brasil, que para este producto es innegociable. Uso la variante de streaming por WebSocket, que empieza a emitir en cuanto tiene el primer fragmento sintetizado en vez de esperar a la frase entera.

**Pipecat (orquestración del pipeline).** Encadena todo lo anterior en un flujo de fotogramas de audio y texto, pasando de una etapa a la siguiente sin ir acumulando latencia por el camino.

**FastAPI y SQLite (servidor y persistencia).** Exponen el contrato con la aplicación móvil — crear sesión, autenticación, historial, traducción de palabras— y guardan usuarios y sesiones.

**React Native con Expo (aplicación móvil).** Un solo código para iOS y Android, con acceso nativo al micrófono y al SDK de LiveKit.

## Metodología

**1. El pipeline de voz.** Cada turno funciona así: el micrófono captura la voz y la manda por WebRTC; Deepgram transcribe en streaming y detecta el fin de la intervención; el texto se añade al historial y va al modelo; la respuesta se sintetiza en voz y vuelve al usuario por el mismo canal. En paralelo, un canal de datos publica las transcripciones para pintarlas en pantalla.

**2. Diseño del prompt del tutor.** El comportamiento del tutor lo define un *system prompt* parametrizado por nivel (A1 a C2, según el Marco Común Europeo de Referencia) y por tema de conversación. Le puse tres restricciones que salen del medio, no del contenido:

- **Máximo dos frases por respuesta.** Todo esto se va a leer en voz alta, y una respuesta larga rompe el ritmo del diálogo además de multiplicar el coste de síntesis.
- **Texto plano, sin formato.** El sintetizador leería los símbolos de marcado en voz alta.
- **Formato fijo de corrección.** El tutor antepone las correcciones gramaticales con una marca reconocible; el servidor la separa del resto de la respuesta y la manda como campo aparte, para que la aplicación pueda mostrarla de otra forma. Esta pieza, aparentemente menor, me dio después dos disgustos que cuento en el apartado 5.

**3. Arquitectura multiidioma: cada idioma es una configuración, no una copia del proyecto.** Agrupo en un perfil todo lo que cambia de un idioma a otro: prompt del tutor, instrucciones por nivel, etiquetas de tema, código de reconocimiento de voz y voces disponibles. Añadir un idioma es escribir ese perfil y registrarlo, sin tocar el motor. Portugués está operativo; francés, italiano e inglés tienen el perfil completo y esperan a que les asigne voces.

**4. Control de concurrencia.** Si el usuario interrumpe mientras el tutor está generando, las peticiones se solapan y la API del modelo devuelve errores de límite de tasa. Puse un mecanismo que serializa las llamadas y se queda solo con la intervención más reciente, que es la que refleja lo que el usuario quiere decir de verdad.

**5. Instrumentación desde el diseño.** Un presupuesto de latencia que no se mide no sirve para nada. El sistema registra, en cada turno hablado, las marcas de tiempo de cada etapa. Eso me permite contrastar el objetivo con el comportamiento real y, cuando un turno sale lento, saber qué componente concreto lo causó.

## **Nota sobre el método de desarrollo**

Trabajé con varias herramientas de inteligencia artificial sobre el mismo repositorio, coordinándolas mediante un documento de traspaso que permite a cualquiera de ellas retomar el trabajo sin contexto previo. Yo dirigí el desarrollo, tomé las decisiones de producto y probé personalmente cada versión.

Menciono el método porque es el objeto mismo de esta formación, y porque dio un resultado que merece la pena: una de las herramientas encontró un fallo de seguridad en código escrito por otra. Lo detallo en las conclusiones.

---

# Apartado 3 — Implementación y validación

## Proceso de desarrollo

Construí el sistema en cuatro fases, y la tercera fue la que de verdad cambió el producto.

**Prototipo del pipeline de voz.** Encadené transporte, transcripción, modelo y síntesis hasta conseguir un turno completo de conversación. Aquí no había métricas todavía: el objetivo era comprobar que las cuatro piezas encajaban.

**Instrumentación.** Antes de optimizar nada añadí el registro de tiempos por etapa. Cada turno hablado deja una línea con las marcas de cada tramo, lo que permite contrastar el presupuesto del apartado 2 con lo que ocurre de verdad y, cuando un turno sale lento, saber qué componente lo causó. Esta decisión es la que hace posible todo lo que viene después.

**Puesta en producción y primera conversación real.** Despliegue en un servidor propio con Docker y Caddy, y primera conversación contra los servicios reales. Aparecieron cinco fallos en una sola sesión, todos de configuración, versiones o permisos, ninguno de arquitectura.

**Ciclo de medición y corrección.** A partir de ahí el trabajo dejó de ser construir y pasó a ser medir, encontrar el fallo, repararlo y volver a medir. Los tres intentos que costó arreglar la toma de turno, que cuento más abajo, son el mejor ejemplo de para qué sirve esto.

El proyecto tiene hoy **318 pruebas automáticas** (205 del servidor y 113 de la aplicación) que se ejecutan en integración continua con cada cambio.

## Métricas de éxito

Definí dos indicadores cuantitativos antes de empezar a medir.

**Latencia percibida.** El tiempo desde que dejo de hablar hasta que empiezo a oír la respuesta. El objetivo, fijado en el apartado 2, era no pasar de 2.500 ms.

**Conversación sin interrupciones.** La proporción de respuestas del tutor que no llegan a oírse porque el sistema dio mi turno por terminado antes de tiempo y yo retomé la frase. Es un indicador que no había previsto: nació de un problema que sólo apareció al hablar con la aplicación de verdad.

Ambos se miden con herramientas incluidas en el repositorio (`scripts/latency_report.py` y `scripts/turn_report.py`), de modo que cualquiera puede reproducir las cifras a partir de los registros en bruto.

## Resultados de latencia

Sobre **170 turnos hablados completos, repartidos en 15 sesiones:**

| Etapas                                  | Mediana         | P90             | Objetivo        | Cumple    |
|-----------------------------------------|-----------------|-----------------|-----------------|-----------|
| Transcripción → primer token del modelo | 1.457 ms        | 2.034 ms        | 500 ms          | No        |
| Primer token → primer audio             | 489 ms          | 1.195 ms        | 800 ms          | Sí        |
| <b>Latencia percibida total</b>         | <b>1.593 ms</b> | <b>2.810 ms</b> | <b>2.500 ms</b> | <b>Sí</b> |

**El 85,9 % de los turnos entra dentro del presupuesto de 2,5 segundos.**

El objetivo global se cumple con holgura en la mediana, pero el reparto por etapas que había escrito estaba equivocado, y la medición lo demostró de una forma que no esperaba.

**El reconocimiento de voz no consume presupuesto: lo devuelve.** Le había asignado hasta un segundo por parecer la etapa más lenta. En realidad la transcripción está lista **352 ms antes** de que el turno se dé por cerrado, porque trabaja en streaming mientras hablo. Ese adelanto es tiempo que el resto del sistema se puede gastar.

**El cuello de botella es el modelo de lenguaje.** Con 1.457 ms de mediana triplica los 500 ms que le había asignado, y es responsable de prácticamente toda la latencia percibida. Elegí un modelo rápido y económico precisamente por esto (apartado 2) y aun así es la etapa dominante.

La conclusión práctica es que optimizar la transcripción no habría servido de nada, que es exactamente donde habría invertido el esfuerzo sin instrumentación.

## Evidencias del impacto: la toma de turno

Este es el hallazgo principal de la validación, y llegar a él costó tres intentos. Lo cuento entero porque los dos primeros fracasos enseñan más que el acierto final.

**El problema.** Al hablar con el tutor me cortaba constantemente: yo hacía una pausa buscando la palabra, el sistema daba mi turno por terminado, el tutor empezaba a hablar y yo retomaba la frase, con lo que su respuesta se descartaba a medias. Medí mis pausas en el registro: **3,6 y 5,0 segundos** dentro de una misma frase.

**Primer intento: la capa equivocada.** Subí el parámetro `endpointing` de Deepgram de 2.000 a 3.200 ms. No cambió nada, y el registro dijo por qué: el turno se seguía cerrando 370 ms después de cada frase. Ese parámetro decide cuándo un *transcrito* es definitivo, no cuándo termina el *turno*.

**Segundo intento: la pieza muerta.** Localicé al responsable real, un modelo semántico de toma de turno (Smart Turn v3) con un tope propio de 3 segundos, y lo configuré con márgenes de entre 3,5 y 6 segundos según el nivel. Tampoco cambió nada. La evidencia fue concluyente:

```

Loading Local Smart Turn v3 model...    ← cargado
[bot] turn patience: 5.0s (level B1)    ← con mi configuración
End of Turn result: ...                  ← 0 veces en toda la conversación

```

**Cero predicciones.** El modelo estaba cargado, había aceptado mis parámetros y no se ejecutaba nunca. El transporte de LiveKit no incorpora detector de actividad de voz en esta versión de Pipecat, así que nunca se emitían los eventos de inicio y fin de habla que despiertan al analizador. Estaba encendido y ciego.

**Tercer intento: darle oídos.** Añadí un detector de actividad de voz (Silero) que alimenta al analizador. A partir de ahí el modelo empezó a trabajar.

## Antes y después

|                                            | Antes     | Después                |
|--------------------------------------------|-----------|------------------------|
| Predicciones del modelo de turno           | 0         | <b>82</b>              |
| Tope de espera                             | 3 s, fijo | decidido por el modelo |
| Espera real durante una duda (mediana)     | —         | <b>7,9 s</b>           |
| Espera máxima observada                    | —         | <b>26,8 s</b>          |
| Esperas por encima del tope antiguo de 3 s | —         | <b>10 de 13</b>        |

En la conversación de validación —**74 turnos míos a lo largo de nueve minutos**— el modelo emitió 50 veredictos de «frase inacabada» frente a 32 de «frase terminada». Se ve funcionando en el registro: mientras yo decía «*Refetivamente,*» «*Europa,*» «*não só em Portugal,*» «*se não*» en trozos sueltos, el modelo dictaminó INCOMPLETE cuatro veces seguidas y me dejó acabar. Con el sistema anterior habrían sido cuatro cortes. El tutor respondió corrigiendo **la frase entera**, reconstruida a partir de los cuatro fragmentos.

## Una advertencia sobre la comparación

El indicador de respuestas descartadas subió del 23 % al 50 %, y presentarlo como un empeoramiento sería deshonesto, porque **después de la reparación ya no mide lo mismo**.

Antes contaba una sola cosa: el sistema me cortaba. Ahora mezcla esa con otra distinta y legítima: que yo retome la palabra tras una pausa que sonaba terminada y el sistema descarte una respuesta que ya no venía a cuento, que es justo lo que hace una persona cuando la interrumpen. Esa segunda situación antes no podía darse, porque no había nada esperando.

Por eso el indicador válido para el antes y el después es el tiempo que el sistema aguanta mientras el alumno duda, no el recuento de descartes.

## Validación cualitativa

Dentro de la propia conversación de nueve minutos, sin tocar ningún parámetro:

|               | Entregadas | Descartadas |      |
|---------------|------------|-------------|------|
| Primera mitad | 7          | 11          | 61 % |
| Segunda mitad | 11         | 7           | 39 % |

Mi impresión al hablar coincide con la cifra: «era cuestión de coger el ritmo de pregunta y respuesta, y al poco tiempo la conversación fluía sin interrupciones».

Esto merece una lectura honesta en las dos direcciones. Por un lado, el sistema pasó a ser utilizable para una conversación adulta y sostenida. Por otro, **parte de la mejora la puse yo adaptándome a la máquina**, y un usuario nuevo no sabrá hacer eso: se encontrará el primer minuto malo y puede abandonar ahí. El problema se ha desplazado del mecanismo a los primeros compases de uso, que es donde apunta la propuesta de paciencia adaptativa del apartado 5.

## Otras evidencias de la corrección

Estos fallos aparecieron en conversaciones reales y se detallan en el apartado 5. Los recojo aquí porque comparten un mismo patrón: ninguno produjo un error en los registros y ninguno se habría encontrado sin hablar con la aplicación.

| Fallo observado                                                                     | Efecto sobre el alumno                          | Estado                                                                                     |
|-------------------------------------------------------------------------------------|-------------------------------------------------|--------------------------------------------------------------------------------------------|
| El tutor «corregía» erratas de la transcripción                                     | Se le reprochaban errores que no había cometido | Corregido                                                                                  |
| «Alentejo» transcrito como «1 entejo» → el tutor corrigió a «um enterro»            | La conversación descarriló                      | Corregido: ahora pregunta                                                                  |
| Marca de corrección escrita en español                                              | El sintetizador la leyó en voz alta             | Corregido en dos capas                                                                     |
| El tutor preguntaba el nombre en cada sesión                                        | Repetición y falta de continuidad               | Corregido                                                                                  |
| Una conexión caída dejaba su error sobre la sesión siguiente                        | Aviso de error sobre una sesión correcta        | Corregido                                                                                  |
| Interrumpir al tutor a media respuesta lo dejaba mudo: el turno no se cerraba nunca | La conversación se paraba sin aviso             | Corregido: una red de seguridad cierra el turno pasados 2 s más que la paciencia del nivel |

## Alcance de la validación

Conviene delimitar qué demuestran estas cifras. Miden **el comportamiento del sistema**: latencia por etapa, toma de turno y ausencia de defectos observables. No miden aprendizaje, y no pueden hacerlo, porque las 15 sesiones son de un único usuario, que además soy yo. Cualquier afirmación sobre eficacia didáctica exigiría el estudio con varios estudiantes que propongo en el apartado 5.

---

# Apartado 4 — Vinculación con competencias adquiridas

Conviene decir de entrada con qué profundidad he seguido la formación: he pasado por casi todas las aulas, en unas a fondo y en otras viendo una parte de las clases. Lo que recojo aquí no es un certificado de finalización, sino aquello que efectivamente he aplicado en el proyecto, con el elemento concreto del sistema que lo justifica en cada caso. Un vínculo que no pudiera señalar dentro del código no lo he puesto.

## Lo que ha sostenido el proyecto

**Aula Audiovisual y Creación.** Es la que más me ha interesado y la que más peso tiene en el producto, porque en una aplicación para hablar **la voz es el producto**. De aquí sale la elección del proveedor de síntesis por la existencia de voces auténticas de portugués de Portugal y no de Brasil, que era un requisito irrenunciable; la comparación entre modelos de generación buscando el equilibrio entre calidad y velocidad; la decisión de usar la variante de streaming por WebSocket, que empieza a emitir audio con el primer fragmento sintetizado en lugar de esperar a la frase completa; y la selección de tres voces con carácter distinto para que el alumno elija con quién quiere hablar. También la producción del vídeo de demostración que acompaña a esta memoria.

**Aula LLMs.** El tutor está gobernado por un modelo de lenguaje (Claude Haiku). De aquí proviene el diseño de los *system prompts*, parametrizados por idioma y por nivel del Marco Común Europeo de Referencia; la restricción de respuesta a dos frases mediante el límite de tokens de salida; y el criterio para elegir un modelo rápido y económico frente a uno mayor, dado que hablando la latencia manda sobre la sofisticación de la respuesta.

Es también donde aprendí algo que sólo se ve al usarlo: que a un modelo no le basta con que le prohíbas, sino que hay que decirle qué hacer en su lugar. Al restringirle los datos legales cambiantes empezó a devolver respuestas vacías, y sólo se corrigió al prescribirle la frase con la que debía salir del paso.

**Aula Bots.** El tutor es un bot conversacional en sentido estricto: un proceso autónomo que se une a la sala como un participante más, mantiene el estado de la conversación y gestiona el turno de palabra. Incluye control de concurrencia para que dos peticiones al modelo no se solapen cuando el usuario interrumpe.

**Aula Developers.** Construcción completa del sistema: servidor en Python con FastAPI, aplicación móvil en React Native con Expo, **318 pruebas automáticas**, control de versiones con Git y revisión de código.

**Aula Fundamentos Técnicos.** Arquitectura del sistema y sus límites, diseño de la API REST, contenerización con Docker y despliegue en servidor propio con proxy inverso y HTTPS.

**Aula Análisis de Datos.** Instrumentación del sistema para medir la latencia por etapas, y análisis de los resultados (mediana y percentil 90) contrastados con el presupuesto declarado como requisito no funcional. Es la competencia que más rendimiento ha dado: sin ella, dos reparaciones que parecían razonables habrían pasado por buenas sin arreglar nada, como se detalla en el apartado 3.

**Aula Automatizaciones.** Integración continua en GitHub Actions que ejecuta la comprobación de tipos y la batería de pruebas con cada cambio, y generación automática del informe de métricas.

## Aportación complementaria

**Aula Webs.** Diseño de la interfaz: sistema de tokens de color derivado en espacio OKLCH, verificación de contraste y maquetación de las pantallas.

**Aula SAAS y Aula Triple A.** Estudio de viabilidad: coste marginal por sesión frente al precio de una clase de conversación, análisis de la competencia directa y delimitación del nicho (portugués europeo frente al brasileño, que es el que ofrecen por defecto las alternativas).

**Aula Freelance.** Entrega profesional: alcance por fases, documentación de traspaso para que otra persona pueda continuar el trabajo sin contexto previo, y valoración económica.

## Nota de método

El desarrollo se apoyó en varias herramientas de IA trabajando sobre el mismo repositorio, coordinadas mediante un documento de traspaso. Durante el proceso, una de ellas detectó un fallo de seguridad en código generado por otra: la función de guardado de sesiones permitía que un usuario sobrescribiera las de otro. Se corrigió con una escritura atómica que comprueba la propiedad del registro, y se cubrió con pruebas específicas.

---

# Apartado 5 — Conclusiones y proyección futura

## Lecciones aprendidas

### La arquitectura aguantó; lo que faltaba era enfrentarla a la realidad

Diseñé y construí el sistema entero antes de que sostuviera su primera conversación de verdad. El día que por fin lo puse a funcionar contra los servicios de producción salieron **cinco fallos en una sola sesión**: una dependencia que había eliminado un parámetro al cambiar de versión, una clave de API que en realidad era su identificador, un tokenizador sin permisos de lectura dentro del contenedor, un analizador de respuestas demasiado estricto y una etapa mal instrumentada.

Ninguno era de arquitectura. Eso me tranquilizó, pero la lección iba por otro lado: aplazar la primera prueba real sale caro, y el precio sube con lo que llevas construido. Los cinco fallos aparecieron a la vez, enredados unos con otros, y separarlos costó bastante más de lo que habría costado encontrarlos de uno en uno.

### Los presupuestos escritos de antemano se corrigen midiendo

En el reparto de latencia del apartado 2 le di hasta un segundo al reconocimiento de voz, porque intuitivamente parecía la etapa más lenta. Cuando instrumenté el sistema me encontré con lo contrario: la transcripción termina unos 350 ms **antes** de que el turno se dé por cerrado, así que no consume presupuesto ninguno. El cuello de botella está en el modelo de lenguaje, que se come con creces el medio segundo que yo le había asignado.

No creo que fuera un error de cálculo, sino de método: escribí el presupuesto antes de tener con qué comprobarlo. Su utilidad real fue obligarme a instrumentar el sistema para verificarlo, y fue ahí donde apareció el dato que lo desmentía.

### En una tubería por capas, acertar la capa importa más que afinar el número

El fallo que más me enseñó fue una interrupción constante: el tutor tomaba la palabra mientras yo seguía buscando cómo acabar la frase. Lo primero que hice fue subir el parámetro `endpointing` de Deepgram, de dos segundos a más de tres.

No cambió nada, y los registros lo dejaron claro: el turno se seguía cerrando 370 ms después de cada frase. Resulta que `endpointing` decide cuándo un **transcrito** es definitivo, mientras que el fin del **turno** lo decide otra cosa distinta, un modelo semántico de toma de turno (Smart Turn v3) que trae su propio tope de tres segundos.

Dos parámetros con nombres parecidos, en capas diferentes, gobernando cosas diferentes. Sin la medición, el cambio habría parecido de lo más sensato y lo habría dado por bueno.

## **Un test puede estar en verde mientras la funcionalidad está muerta**

Añadí que el tutor reconociera al alumno por su nombre. Las pruebas pasaron, desplegué, y el tutor siguió preguntándome quién era. El test comprobaba que la función que lanza el bot recibía bien los datos, pero nadie comprobaba **la llamada real**, que seguía tirando de los valores por defecto.

La pieza estaba bien probada. La costura entre piezas, no. Y un test que verifica el componente pero no dónde se enchufa da una confianza falsa, que para mi gusto es peor que no tener test.

## **A un modelo de lenguaje no le basta con que le prohíbas: hay que decirle qué hacer en su lugar**

Cuando le puse límites al tutor para que no afirmara datos legales que cambian con frecuencia, empezó a devolver respuestas vacías. Se quedaba callado, sin más. La prohibición dejaba un hueco y el modelo no sabía con qué llenarlo. Sólo cuando le escribí la frase concreta con la que debía salir del paso volvió a comportarse.

## **Un prompt es una petición; un analizador es una garantía**

El tutor tenía que marcar sus correcciones con la palabra portuguesa «Correção». Un día escribió «Corrección», en español. El analizador no reconoció esa grafía, no la recortó, y el sintetizador **se la leyó en voz alta al alumno**.

Lo peor es que no fue un fallo ruidoso: no dejó ni un error en los registros, sólo un tutor diciendo cosas raras. Lo arreglé en dos capas, y el orden de importancia es el contrario al que parece: el prompt insiste en la forma correcta, pero es el analizador el que garantiza que, escriba lo que escriba el modelo, esa marca no llegue nunca al altavoz.

## **Una transcripción mal hecha puede acabar convertida en un reproche al alumno**

El problema de calidad más grave que me encontré no fue técnico. Dije «Alentejo», el reconocedor escribió «1 entejo», y el tutor me anunció muy convencido que lo correcto era «um enterro» —un entierro— y a continuación se negó a seguir la conversación por considerarla un asunto personal delicado.

Un error de máquina se transformó en una corrección falsa contra el alumno y descarriló el diálogo entero. El fondo del asunto es que el tutor trataba la transcripción como si fuera exactamente lo que yo había dicho. Ahora, cuando una palabra no encaja en el contexto, pregunta en lugar de corregir.

## **Una funcionalidad que no se ve no existe**

Desde las primeras versiones se podía tocar cualquier palabra del tutor para consultar su traducción. Está implementada, probada, con caché en el servidor para no pagar dos veces la

misma consulta. Y llevaba semanas usando la aplicación sin descubrirla, hasta que propuse añadir «una especie de diccionario consultable» sin saber que ya lo tenía delante.

La causa era de una sencillez incómoda: las palabras llevaban un subrayado punteado al 33 % de opacidad sobre un fondo casi negro. Técnicamente la señal estaba ahí; para un ojo humano, no.

Ninguna prueba automática podía detectar esto, porque el código funcionaba perfectamente. Y si el autor de la aplicación no encuentra una funcionalidad suya, nadie la va a encontrar. Lo arreglé subiendo el contraste del subrayado y, sobre todo, diciéndolo con palabras encima de la conversación, que era lo que faltaba de verdad.

## **Dudar no es el caso raro: es el caso de uso**

La conclusión que más ha cambiado el producto es también la más simple. El sistema tomaba mis pausas largas por el final de mi turno, y yo llegué a disculparme por hacer pausas mientras buscaba la palabra.

En una aplicación cuyo propósito entero es ayudar a hablar a quien todavía no lo hace con soltura, vacilar no es una desviación de lo esperado: es exactamente lo que va a pasar todo el rato. Los valores que había puesto salían de lo que resulta natural entre dos personas que ya dominan el idioma, y esa era la referencia equivocada desde el principio.

## **Escalabilidad y mejora continua**

### **Ampliación a otros idiomas**

La arquitectura separa lo que cambia entre idiomas —prompt, voces, código de reconocimiento, etiquetas de temas— de lo que no cambia: la tubería de audio, la gestión de turno, el registro de latencias y el almacenamiento. Cada idioma es un perfil de configuración y no una bifurcación del código.

El registro ya contempla francés, italiano e inglés británico, marcados como no disponibles mientras no les asigne identificadores de voz reales y no revise los prompts con hablantes nativos. Las pruebas verifican que los tres comparten las mismas claves de tema y de nivel que el portugués, así que activarlos consiste en rellenar datos, no en reescribir lógica.

Hay una decisión que el código no resuelve y que conviene dejar advertida: el nombre. «Falandu Portugûês» nombra bien la versión portuguesa, pero no sirve para una plataforma que enseñe cuatro idiomas. Cuando llegue ese momento, la aplicación necesitará un nombre genérico propio, y cada idioma conservará el suyo en su propia lengua: Falandu Portugûês, Parlant Français, Parlantu Italiano, Speaking English. Es un cambio de marca, no de arquitectura, y por eso lo dejo aquí y no en el código.

## Validación con usuarios reales

Es la limitación principal del trabajo y prefiero decirlo sin rodeos: **lo he validado con un único usuario, que soy yo**. Las cifras del apartado 3 miden cómo se comporta el sistema, no lo que aprende nadie. Para afirmar cualquier cosa sobre eficacia didáctica haría falta un estudio con varios estudiantes, un grupo de control y una medida de progreso independiente.

El paso siguiente sería una prueba con diez o quince estudiantes durante unas semanas, midiendo minutos hablados por sesión y cómo evoluciona la proporción de correcciones.

## Identidad ligada al aparato

La cuenta se identifica hoy por el identificador que iOS le asigna a la aplicación en cada dispositivo. Lo decidí así a propósito para la fase de pruebas, porque quita de en medio todo el trámite de registro, pero tiene una consecuencia que no es menor: quien cambia de teléfono pierde su historial y no hay forma de recuperarlo.

Lo descubrí de la peor manera, probando la aplicación en un simulador distinto del habitual y encontrándome con que el servidor no me conocía. La ruta de salida está clara: añadir una identidad independiente del aparato, por correo o identificación federada, y dejar el acceso actual como opción de invitado.

## Paciencia adaptativa

Hoy ajusto la tolerancia a las pausas según el nivel declarado, pero dentro de un mismo nivel caben ritmos muy distintos. Y el sistema tiene delante la información que necesitaría para hacerlo mejor: sabe perfectamente cuándo ha interrumpido a alguien, porque esa persona retoma la frase y su propia respuesta se descarta.

Ese descarte es una señal de error que ahora mismo se está desaprovechando. Un ajuste que subiera la paciencia después de cada interrupción detectada convergería al ritmo de cada persona en unos pocos minutos de conversación, sin pedirle que configure nada.

## Observabilidad en producción

El registro de latencias funciona y produce datos por turno, pero dejé el seguimiento de errores (Sentry) instalado y sin activar, y no monté vigilancia de disponibilidad del servicio. Para abrir esto a usuarios reales las dos cosas son requisito previo y no mejora opcional: sin ellas, un fallo en producción sólo se detecta cuando alguien se molesta en quejarse.